

Idea Name: BlastRadius Sim

A What-If Failure Simulator for Live Kubernetes Dependency Graphs

1. Team Details	TEAM
-----------------	------

Team Name: Deshire

Team Members:

- Sona Bhatia
- Caleb Benfield

2. Problem Statement & Relevance	CONTEXT
----------------------------------	---------

Selected Theme: Theme 2 - Beyond Monitoring: AI Agents for Real-Time Pod Resource Discovery and Dependency Mapping

Dependency-mapping and root-cause tools tell engineers what already broke and why, after the fact. They do not tell an engineer what would break next, before they make a change. Every deploy, scaling decision, or node drain is effectively a bet: teams rarely know in advance which services sit on a single point of failure, or how far a failure would actually cascade, until an incident forces them to find out live. This turns routine operational decisions, taking a node down for maintenance, consolidating a service, approving a risky deploy, into guesswork that leans on tribal knowledge of “what talks to what.”

BlastRadius Sim closes this gap by turning the live dependency graph from a diagnostic record into a forward-looking planning tool. Instead of only reconstructing incidents after they happen, an engineer can select any node — a pod, service, or node — and simulate what happens if it fails right now, before committing to the change that might cause it.

Why this problem

- Teams routinely discover single points of failure only when they cause an outage, not before.
- Existing dependency and root-cause tools are reactive by design — they explain incidents, they do not prevent them.
- Risk assessment for deploys, node maintenance, and scaling decisions is usually informal, based on whoever remembers the architecture best.
- SLA/SLO impact of a potential failure is rarely visible until the failure is already in progress.

3. Idea and Approach	SOLUTION
----------------------	----------

Solution Overview

BlastRadius Sim sits on top of a live, continuously updated Kubernetes dependency graph. An engineer clicks any node and runs a simulated failure: the tool walks the graph outward from that node, projects which downstream services and pods would be affected, estimates which SLAs/SLOs would likely breach, and surfaces pre-emptive mitigations (e.g. add redundancy, reroute traffic, delay a planned change). The result is a fast, visual “what-if” answer an engineer can get in seconds, before taking an action — not a report generated after something has already gone wrong.

Key Features

- **Interactive Failure Simulation:** Click-to-simulate failure on any node in the live dependency graph, with results in seconds.
- **Blast Radius Projection:** Projects every downstream service and pod affected by a simulated failure, ranked by proximity and criticality.
- **SLA/SLO Impact Estimate:** Cross-references affected services against defined SLAs/SLOs to flag which commitments are at risk.
- **Suggested Mitigations:** Surfaces concrete pre-emptive actions (add a redundant path, stagger a rollout, reroute traffic) ranked by how much they shrink the projected blast radius.
- **Pre-Deploy Risk Check:** Lets engineers compare blast radius before and after a proposed architecture change, ahead of merging it.
- **Risk Heatmap:** Ranks nodes across the cluster by how large a blast radius their failure would cause, surfacing hidden single points of failure proactively.

4. Technical Solution & Innovation

TECHNICAL

Architecture Overview

- **Discovery & Graph Layer:** Watches the Kubernetes API and sampled network flow data to keep an accurate, real (not just declared) dependency graph.
- **Simulation Engine:** Given a target node, traverses the live graph outward to identify all directly and transitively dependent services.
- **Impact Scoring Layer:** Combines graph centrality with historical incident data to estimate the likely severity and SLA impact of a simulated failure.
- **Mitigation Recommender:** Matches the projected blast radius against a library of mitigation patterns (redundancy, failover, staged rollout) to rank suggested actions.
- **Presentation Layer:** Interactive graph explorer where engineers trigger simulations and compare before/after states.

Model

Type: Graph simulation + learned impact scoring

- Deterministic graph-traversal simulation (BFS/DFS over the dependency graph) projects reachable blast radius from any node.
- A lightweight scoring model estimates severity using graph centrality and historical incident outcomes.
- Mitigation suggestions are matched from a structured pattern library, not freely generated — kept explainable and auditable.

Technical Stack

- **Frontend:** React, for the interactive graph explorer and simulation view.
- **Backend:** Python, FastAPI, orchestrates discovery and simulation.
- **Cluster Integration:** Kubernetes API client, eBPF-based flow sampling, for real dependency data.
- **Graph Database:** Neo4j, stores the live dependency graph.
- [Note: architecture and stack are an initial direction — expected to evolve as the prototype is built out.]

Innovation Highlights

- **Proactive, not reactive:** Shifts dependency graphs from a post-incident diagnostic record to a pre-decision planning tool — answering “what if” instead of only “what happened.”
- **Pre-deploy risk check:** Lets an engineer directly compare the blast radius of an architecture before and after a proposed change, ahead of merging it.

- **Explainable by construction:** Simulation results are grounded in the actual live graph and historical data, so every projection can be traced and verified.
- **Additive integration:** Runs as a read-only overlay on existing dependency-discovery infrastructure, so it can be adopted incrementally.

5. Implementation & Impact

IMPACT

Future Improvements

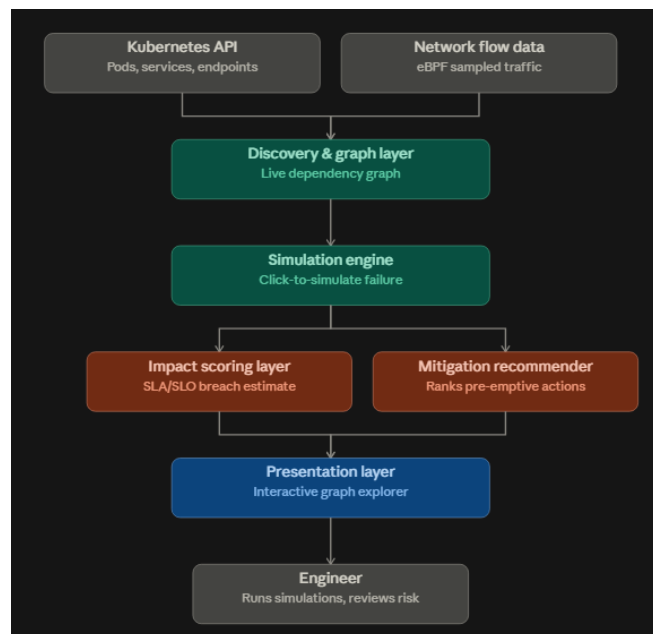
- Extend simulation to multi-node and cascading failure scenarios (simulate two or more simultaneous failures).
- Add automated “risk gate” checks in CI/CD that block or flag deploys projected to significantly increase blast radius.
- Support cross-cluster and multi-cloud simulation as dependency discovery extends beyond a single cluster.
- Introduce a feedback loop where real incident outcomes are used to continually calibrate the impact scoring model.

Expected Business Value

- Fewer outages caused by previously-unknown single points of failure, surfaced proactively instead of after an incident.
- Faster, more confident risk assessment for deploys, node maintenance, and scaling decisions.
- Reduced reliance on individual tribal knowledge of system architecture when planning changes.
- Clearer, data-backed justification for infrastructure investment in redundancy where it matters most.

6. Methodology / Architecture Diagram

ARCHITECTURE



[Note: This is an initial prototype concept. The scope, architecture, and technical stack outlined above are expected to evolve as the idea is developed further.]